

The Scenario

You've just joined a fintech startup as a QA Engineer. On day one you're handed a mobile trading app, for both iOS and Android, that's two weeks from its first public release. There's no existing test suite, no QA documentation, and the dev team has been shipping fast. Assume real user funds are involved.

1. Where do you start?

Answer:-

I would start by understanding the release scope and the risks associated with what is being shipped. I would not begin by randomly exploring the app, especially in a fintech environment where real user funds are involved.

The first step would be to identify the source of truth. That could be user stories, product requirements, designs, API documentation, release notes, or walkthroughs from product and engineering if formal documentation does not exist. From there, I would align with the product and development teams to understand what is in scope, what has changed recently, what remains unstable, and what the business considers critical for release.

Once the scope is clear, I would prioritize testing by risk. Given the tight timeline, I would focus first on flows where failure could impact users, money movement, data accuracy, security, or trust. I would separate release-blocking areas from lower-risk issues so the team has a clear view of what must be fixed before launch and what can be tracked as a known risk.

2. How would you approach testing this app?

Answer:-

With only two weeks left before release, no existing test suite, and devs still shipping fast, I would keep the approach very practical. Let's not aim for ideal QA coverage at this stage. My focus would be to understand what is in scope, what is stable, what is still changing, and where the biggest release risks are.

I would test in a risk-based way. Anything that could affect users, funds, data accuracy, security, compliance, or trust would be tested first and in greater depth. Lower-risk UI or cosmetic issues would still be logged, but they would not take priority over release blockers.

I would also not rely only on the mobile UI. If some parts of the app are still unstable, I would validate the underlying behavior through APIs, logs, or backend responses where possible, so we can catch critical issues earlier.

For mobile testing, I would focus on real-world usage, not just happy paths. I would test both iOS and Android with cases like slow network, app going to background, session expiry, delayed responses, repeated actions, and recovery after failure.

The main goal would be to give the team fast and honest feedback: what has been tested, what is blocked, what is risky, what must be fixed before release, and what risk the business is accepting if they still decide to launch.

3. What does QA look like inside a sprint, from ticket creation through to regression?

Answer:-

Inside a sprint, QA should start from ticket creation, not after development is complete. As soon as a ticket is picked up, I would review the requirement, acceptance criteria, designs, and any backend/API impact. The goal is to understand the expected behavior early and identify gaps before the team spends time building something unclear.

During refinement or sprint planning, I would ask questions around edge cases, dependencies, validations, error handling, and possible impact on existing flows. I would also share an initial test checklist or test scenarios with the developers, so they already know what will be verified. This helps catch logical gaps early instead of finding everything at the end.

While development is in progress, QA should stay close to the dev and product teams. It should not feel like dev vs QA. Everyone is working toward the same release, so communication is important. If the frontend is not ready yet, I would validate backend behavior, API contracts, logs, or service responses where possible, so testing does not have to wait completely.

Once the ticket is ready for QA, I would test it against the acceptance criteria, cover positive and negative scenarios, check edge cases, and verify any impacted areas around it. If defects are found, I would report them clearly with steps, actual result, expected result, environment, screenshots or logs where needed, and severity based on release impact.

After fixes are delivered, I would retest the defect and also check nearby impacted areas to make sure the fix did not break something else. Toward the end of the sprint, I would run focused regression based on what changed in the sprint, especially around critical and high-risk flows. The regression should not be random; it should be based on impact, recent changes, and areas where defects were found.

By the end of the sprint, QA should be able to clearly communicate what was tested, what passed, what failed, what is blocked, what risks remain, and whether the build is ready to move forward.

4. What does your ideal regression suite look like?

Answer:-

My ideal regression suite would be focused, risk-based, and useful for release decisions, not just a long list of test cases. I would maintain a small smoke set for every build to quickly confirm the app is stable enough for testing, and a deeper regression set for the areas that carry the highest business and user risk. Since this is a mobile fintech app, the suite should cover the main journeys, negative cases, data consistency, security-related behavior, and anything that could impact user trust.

I would also ensure regression is not random. It should be based on what changed in the sprint, which areas are connected to that change, and any defects found recently. Over time, any repeated defect or production issue should be added back into regression so the same issue does not recur. The goal of regression is to help the team quickly understand what still works, what broke, what risk remains, and whether the build is safe enough to move forward.

For automation, I would apply it only to stable parts of the app where the behavior is clear and not changing every sprint. The goal would be to reduce repeated manual effort for smoke and regression checks, not to automate everything blindly. Anything unstable or still being actively changed should remain manual until it becomes reliable enough to automate without creating flaky tests.

5. What would keep you up at night about this app specifically and releasing to the public?

Answer:-

What would keep me up at night is anything that could affect user funds, user trust, regulatory compliance, or the company's credibility after release. In a fintech app, a bug is not just a UI issue. If something goes wrong with money movement, data accuracy, user access, failed transactions, duplicate actions, or blocked user journeys, it can directly damage user confidence and create serious business risk.

On the consumer side, I would also be very aware of the regulatory bodies the company is licensed under. Any discrepancy in how transactions, user data, reporting, audit trails, limits, or compliance-related flows behave can have serious consequences. In fintech, poor quality checks not only create a bad customer experience; they can also create regulatory exposure, penalties, loss of trust, or, in extreme cases, even impact the company's ability to operate.

For me, quality in this kind of app is about protecting both the user and the business. The user should feel that their money, data, and actions are handled correctly, while the company should be confident that the product is operating within its regulatory obligations. What would concern me most is releasing with half-tested critical areas and only discovering after launch that we have damaged user trust, created financial risk, or exposed the company to regulatory issues.